

공학도를 위한 창의적 컴퓨팅

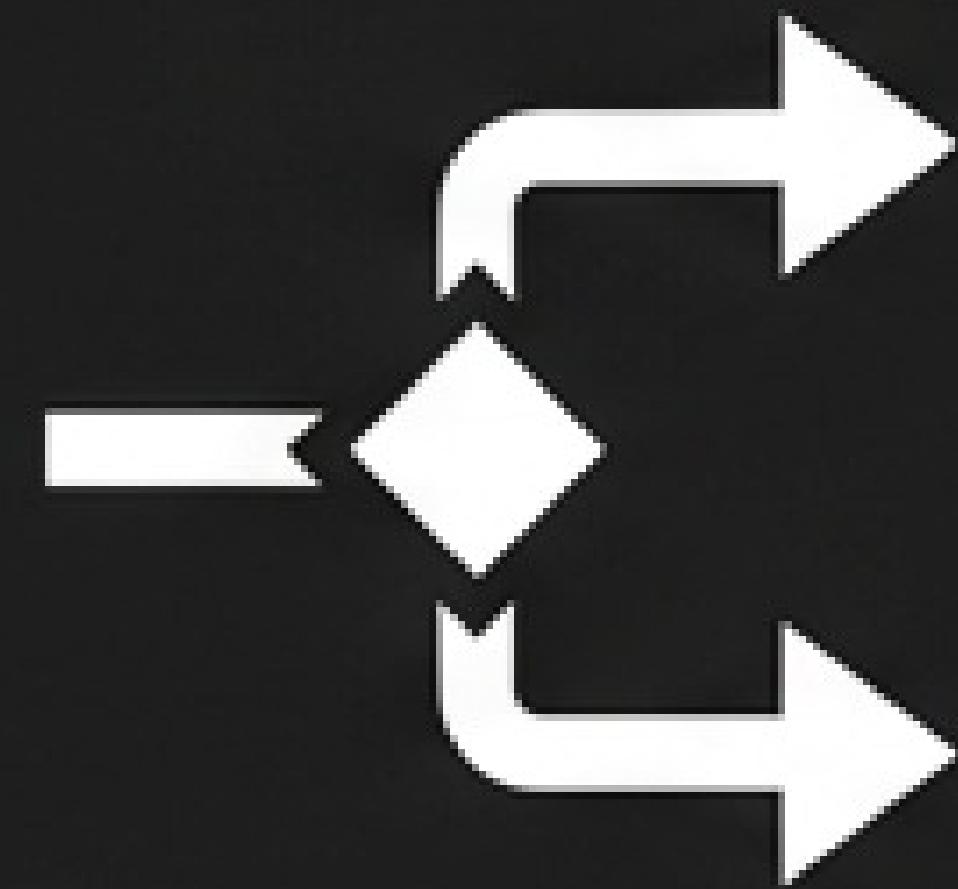
실습 9 : Data Types & Bitwise Operations (데이터 유형과 비트 연산)

실습 목표 (Lab Objectives)

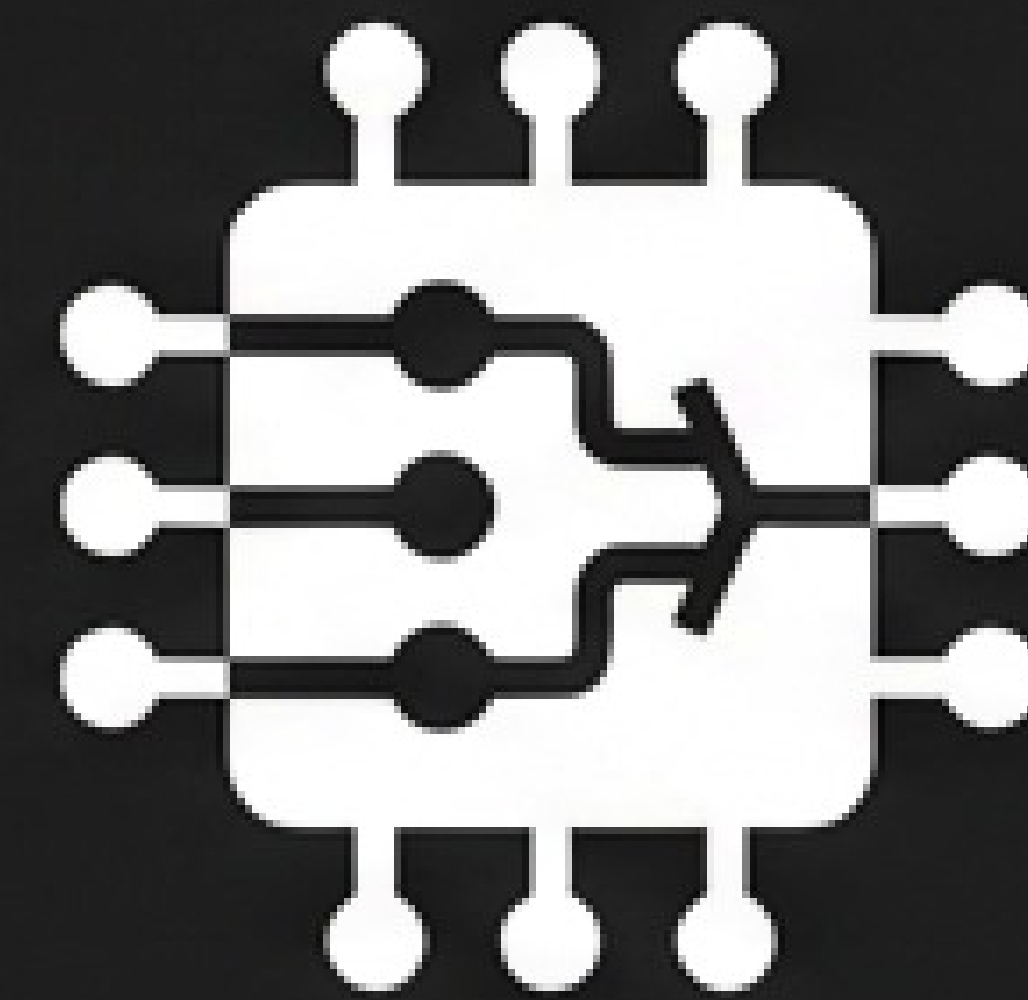
기본 로직을 넘어 데이터의 저장 방식과 출력 형식을 제어합니다.



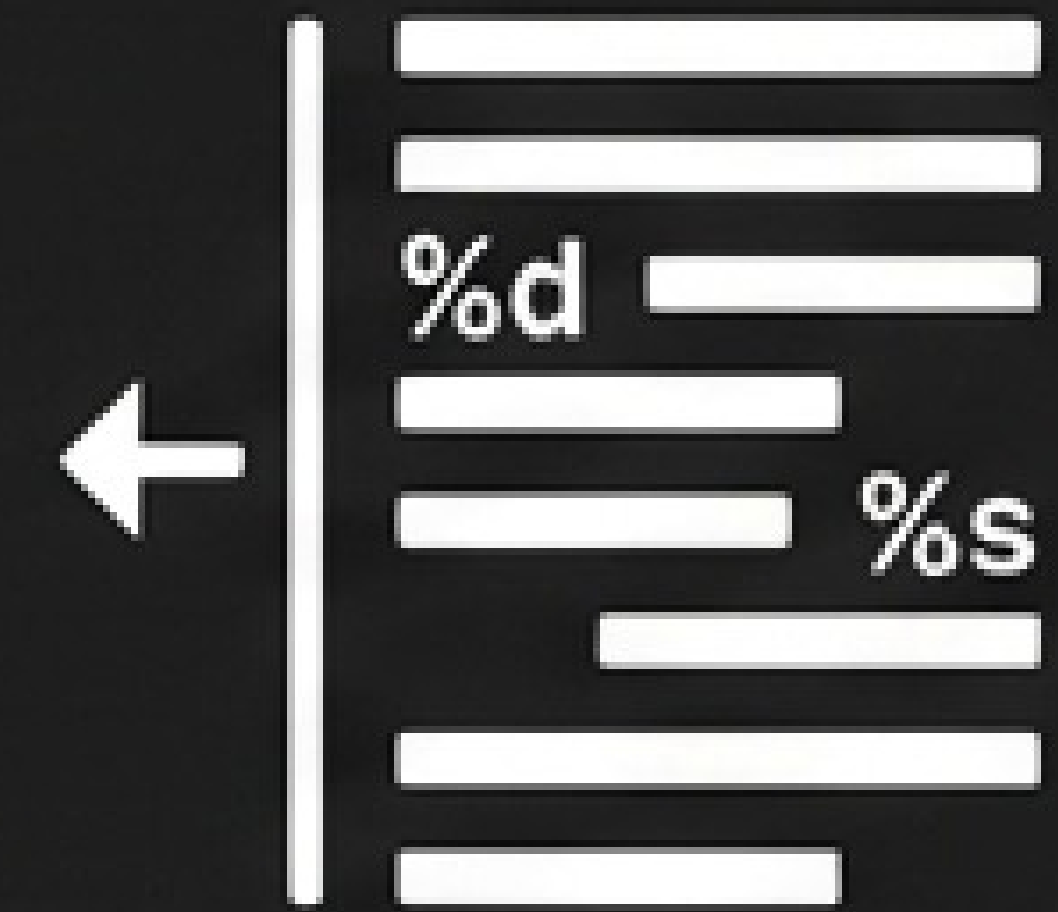
ASCII & Char
문자와 숫자의 관계



Control Flow
조건문의 제어



Bitwise Operations
비트 단위 연산



Formatting
printf 정밀 제어

Concept: ASCII Code



```
char c = 'A';  
printf("%d", c); // Outputs: 65
```

컴퓨터는 문자를 정수(ASCII Code)로 저장합니다. 대소문자 차이(32)를 이용해 변환합니다.

Concept: Bitwise & Formatting

Bitwise Operators

& : AND

| : OR

^ : XOR

~ : NOT

<< : Left Shift (x2)

>> : Right Shift (/2)

0001 << 1 → 0010

Format Specifiers

최소 필드 폭 데이터 타입
(d, f, s 등)

서식 지정자 시작 소수점 이하 자릿수

"%[width][.precision]type"

"%10d" → 10칸 확보 우측 정렬

"%.2f" → 소수점 2자리

비트 단위의 정밀한 연산과 출력 형식을 지정하는 서식 지정자(Specifier)를 학습합니다.

Lab Assignments (실습 과제)



직접 코드를 작성하여 다음 문제들을 해결하세요.

Task 1: Case Converter (대소문자 변환)

문자를 입력받아 대문자는 소문자로, 소문자는 대문자로 변환하여 출력하세요.

Terminal

Input: A

Result: a

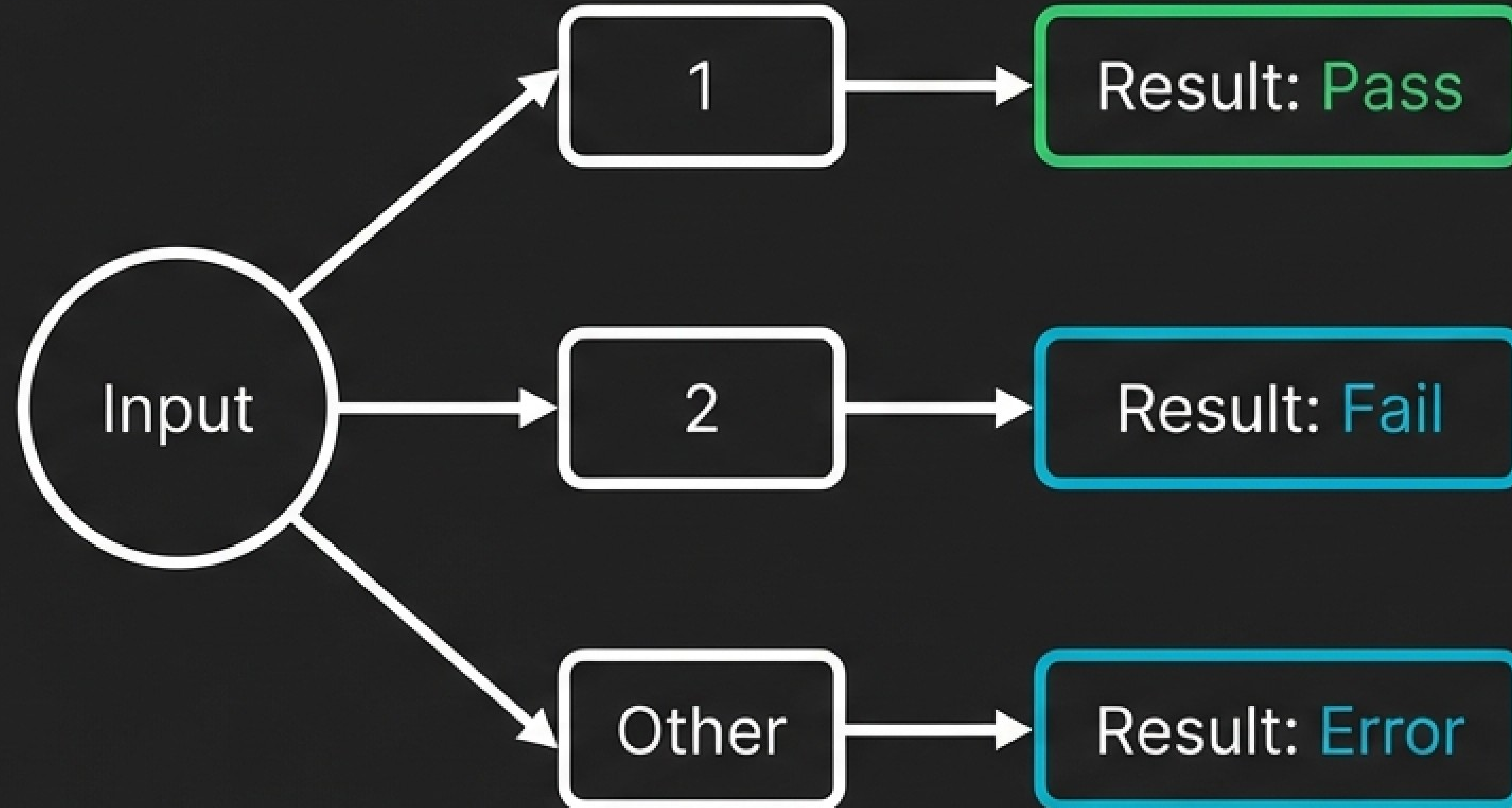
Input: b

Result: B

Hint: ASCII 값의 차이(+/- 32)를 이용하세요. if문을 사용하세요.

Task 2: Pass/Fail Logic

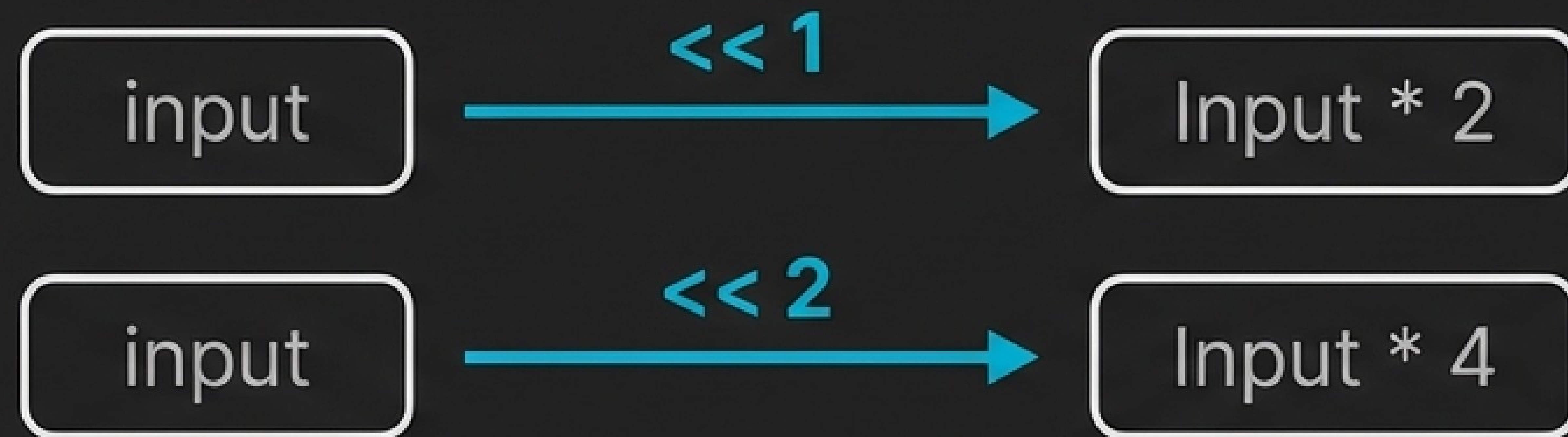
정수 1 또는 2를 입력받아 결과를 출력하세요. (그 외 숫자는 'Error' 출력)



```
switch(input) {  
  case 1: ...  
  case 2: ...  
  default: ...  
}
```


Task 3: Bitwise Operations

정수를 입력받아 비트 이동 연산자(<<)를 사용하여 2배, 4배 값을 계산하세요.



```
Input number: 10  
x2 (using << 1): 20  
x4 (using << 2): 40
```



Note: 곱셈 기호(*)를 사용하지 말고 비트 연산자를 활용하세요.

Task 4: Output Formatting

실수(float)를 입력받아 다음 서식에 맞춰 출력하세요.

1. 소수점 둘째 자리까지 출력
(%.2f) 

2. 전체 폭 10칸을 확보하고
우측 정렬 (%10.2f) 

Input float: 3.141592

Round:

3	.	1	4						
						3	.	1	4

Align:

						3	.	1	4